Author's accepted manuscript. Accepted for publication in the Journal of Information and Data Management (JIDM), 2026. The final version will be available at https://journals-sol.sbc.org.br/index.php/jidm.

Journal of Information and Data Management, 2026, ??:?, doi: 10.5753/jidm.2026.XXXXXX
This work is licensed under a Creative Commons Attribution 4.0 International License.

Enabling Collaboration in Scientific Experiments

Eduardo Jandre [**Universidade Federal Fluminense** | ***eduardojandre@gmail.com***]
Arthur Paiva [**Universidade Federal Fluminense** | ***arthurpaivasg@yahoo.com.br***]
João Felipe Nicolaci Pimentel [**Universidade Federal Fluminense** | ***joaofelipe@ic.uff.br***]
Bruna Diirr [**Universidade Federal do Estado do Rio de Janeiro** | ***bruna.diirr@uniriotec.br***]
Vanessa Braganholo [**Universidade Federal Fluminense** | ***vanessa@ic.uff.br***]

Instituto de Computação, Universidade Federal Fluminense, Av. Gal. Milton Tavares de Souza, s/n, São Domingos, Niterói, RJ, 24210-590, Brazil.

Received: 12 September 2025 • **Accepted:** 27 July 2026 • **Published:** DD Month YYYY

Abstract Science is a collaborative activity. The findings we achieve today are only possible due to the accumulated knowledge constructed by several generations of scientists. In addition, the way scientists do science has changed with technological advances: it has increasingly become computer-based. Moreover, collaboration in science has increased over time, with teams instead of individuals conducting more and more scientific research and teams growing in size. These new ways of doing research demand proper tooling support to help attribute the correct credit to the scientists who worked on the experiment, identifying implicit collaborations (when a scientist uses data produced by another experiment), and collecting and consolidating experiment data generated by different users and devices. However, although there are tools to support computer-based experiments through provenance, they either are (a) workflow management systems, which require the experiment to be implemented as a workflow in a particular format, or (b) script-based systems, which still have several limitations in supporting collaboration. In this article, we focus on the limitations of script-based systems to support collaboration. Most of these systems do not provide collaborative features, demanding that people involved in the research collaborate outside of the tool. This causes loss of important provenance information and makes the research tasks harder. To reduce such limitations, in this article, we propose an extension to noWorkflow, a script-based provenance system, to add collaborative capabilities such as: sharing executed trials among scientists, thus providing a single provenance database representing the whole experiment; creating the concept of "Experiment" in the provenance storage, thus allowing scientists to have multiple experiments in a single provenance storage; and enriching collaboration in the context of different experiments. We present the results of a validation experiment, which helps to understand how the proposed approach increases collaboration and productivity, besides enriching the collected provenance information.

Keywords: Collaboration, eScience, Provenance, Scientific Experiments

1 Introduction

Science, as we know, is a result of centuries of hard and collaborative work. Millions of people from around the globe contributed to the accumulated knowledge we have today. Although it is common that a single "genius" receives credit for some advances, the truth is that, in general, these "geniuses" used information that was only possible due to the effort of other people to acquire and document it [Sawyer, 2017; Bennis and Biederman, 2007]. Thus, science is a collaborative activity of humankind.

Collaboration in science occurs not only in the reuse of knowledge acquired by other researchers but also in ongoing research. Research is usually carried out by multiple scientists working together [Caldwell and Lindberg, 2018]. This collaboration can occur in different intensities, reflected in publications with multiple authors or acknowledgments to other researchers. Wuchty *et al*. [2007] analyzed millions of publications, concluding that collaboration in publications has increased in the last decades. The number of publications made by teams has increased, and the size of the teams has grown. Additionally, when analyzing the impact of the teams' publication when compared to single-author publications, Wuchty *et al*. [2007] concluded that publications produced by teams usually receive more citations on average, even when self-citations are ignored. Hence, collaboration is often encouraged and even demanded by some funding agencies [Sonnenwald, 2007].

At the same time, some technological advances help to explain this growing collaborative behavior. Computer technology has evolved rapidly; computers, as we know, were conceived and have increasingly become more accessible. Also, the internet was created and spread all around the world [Tanenbaum, 2002]. This evolution has derived two side effects: (i) it allowed new types of scientific experiments to be performed (*in virtuo*, experiments involving the interaction among participants and a computational model of the reality, and *in silico*, experiments characterized for both the subjects and real-world being described as computer models) [Travassos and Barros, 2003]; and (ii) they allowed new ways of collaboration, making collaboration easier to be carried out by people located anywhere on Earth.

In silico experiments are executed on the computer, where both the execution and observation environments are entirely composed of computational models for which no human interaction is foreseen. They usually require much more support from data management tools and software engineering

when compared to traditional experiments [Travassos and Barros, 2003]. While the scientific community has increasingly adopted this kind of experiment, it poses new challenges in terms of collaboration, such as: attributing the correct credit to the scientists who worked on the experiment; identifying implicit collaborations (for example, when a scientist uses data produced by another experiment); and collecting and consolidating experiment data generated by different users and devices [Sonnenwald, 2007; Altintas *et al*., 2011a]. Provenance [Herschel *et al*., 2017; Freire *et al*., 2008] can help solve some of these problems.

Jandre *et al*. [2020] present a survey on provenance in collaborative in silico scientific experiments. In the survey, the authors conclude that provenance collection is a common feature in many workflow management systems [Freire *et al*., 2006; Zhang, 2010] and script-based systems [Murta *et al*., 2014; Davison, 2012; Miao *et al*., 2017; Servillat *et al*., 2021; Köhler *et al*., 2024; Salazar *et al*., 2021], which have been a popular way of running in silico experiments. However, collaboration is still one of the challenges in the area [Davidson and Freire, 2008; Herschel *et al*., 2017; Jandre *et al*., 2020].

In fact, as also pointed out by Jandre *et al*. [2020], the collaboration support of script-based solutions is still quite limited. Although some authors [Zhang *et al*., 2012; Freire *et al*., 2006; Ellkvist *et al*., 2008; AbediniAla and Roy, 2022; Coelho *et al*., 2022; Han *et al*., 2024] deal with provenance-aware tools to assist the design and analysis of experiments collaboratively, they are based on workflow management systems, ignoring the fact that many scientists use scripts in their experiments [Pimentel *et al*., 2019]. Some scientists use standard software development tools like version control systems to collaborate and execute the experiment. However, these tools typically do not address specific problems of scientific research, such as provenance gathering [Jandre *et al*., 2020]. Those scientists usually manually annotate important provenance information about the experiment, which is error-prone.

Tools that allow capturing the provenance of scripts have limited support for collaboration, demanding people involved in the research to collaborate outside of the tool, which causes loss of important provenance information: the provenance about the collaboration itself. This kind of provenance is essential for correctly attributing credit to researchers who worked on the experiment, identifying implicit collaborations (when a scientist uses data produced by another experiment), and collecting and consolidating experiment data generated by different users and devices.

To fill in this gap, this work proposes a provenance-aware approach that assists scientists in conducting in silico collaborative experiments. For this, we take advantage of existing research on conducting experiments that are materialized as scripts. Among the existing approaches, we chose noWorkflow [Murta *et al*., 2014; Pimentel *et al*., 2017], a tool that captures provenance of Python scripts. The choice was made because noWorkflow allows capturing provenance transparently. This means that, unlike other tools in the literature (such as YesWorkflow [McPhillips *et al*., 2015], Komadu [Suriarachchi *et al*., 2015], DfAnalyzer [Silva *et al*., 2020]) noWorkflow collects provenance without requiring the scientist to instrument the code. Furthermore, noWorkflow is Open Source[1]. noWorkflow also supports two granularity levels: *coarse-grained*, which captures function activations (calls, parameters, and return values) and file accesses; and *fine-grained* (the default), which additionally captures local variable evaluations and data dependencies among variables. We extend noWorkflow by adding collaborative capabilities such as: sharing executed trials between scientists, thus providing a single provenance database representing the whole experiment; and creating the concept of "Experiment" in the provenance storage, thus allowing scientists to have multiple experiments in a single provenance storage.

To validate our approach, we conducted two simulated experiments. The first one is aimed at evaluating how our approach increases the productivity and fluidity of a collaborative scientific experiment. We contrast our approach with the alternative of using the original version of noWorkflow as a baseline. The results showed that our approach allows scientists to answer questions about the experiment that were not possible when using noWorkflow's original version. In the second evaluation, we show how we are able to answer provenance queries obtained from the First Provenance Challenge over a provenance database that was generated for a simulated collaborative experiment. To be able to do that, we contribute by proposing a translation of those queries (that were initially targeted at workflow systems) to experiments written as scripts.

The remainder of this article proceeds as follows. Section 2 details a survey we made to quantify and better understand the importance of collaboration in science and how new approaches and tools could help. Section 3 presents an analysis and evaluation of the current work addressing support for script-based experiments and collaboration on in silico experiments. Section 4 presents noWorkflow, how it works, and why and how it helps scientists with script-based experiments. Section 5 describes our proposed approach for enabling collaboration in scientific experiments. Section 6 presents the experimental evaluation and discusses the obtained results. Finally, Section 7 concludes this article.

2 Survey on Collaboration in Science

To better understand the importance of collaboration in the day-to-day lives of scientists and how the existing approaches could be improved, we conducted a survey with members of the scientific community. The survey was conducted 100% online, and all responses were anonymous. We ended up with 19 respondents, which gave us an initial idea of their reality and the problems they face. The survey has a total of 15 questions: six closed questions and nine discursive questions.

For the purpose of this article, we will not cover the whole survey. Instead, we concentrate on seven main questions that are directly connected with the main subject of this article. These questions cover: respondents' characterization (3 questions), tools they use in their scientific experiments (1 question), size of the teams in their experiments (1 question), and difficulties related to collaboration in their experiments (2 questions).

[1] `https://github.com/gems-uff/noworkflow`

Table 1. Survey results.

	Question	Answer	%	#
Q1	Educational Level	Undergraduate degree	5.3 %	1
		Master's in progress	5.3 %	1
		Master's degree	5.3 %	1
		PhD in progress	15.8 %	3
		PhD degree	68.4 %	13
Q2	# of Experiments	1	15.8 %	3
		5 to 10	5.3 %	1
		More than 10	78.9 %	15
Q3	Years of Experience	Less than 1 year	10.5 %	2
		Between 2 and 5 years	15.8 %	3
		Between 5 and 7 years	15.8 %	3
		Between 7 and 12 years	26.3%	5
		More than 12 years	31.6%	6
Q4	Preferred Tools	Ruby	5.3 %	1
		SciCumulus	5.3 %	1
		Spark	5.3 %	1
		Spark/Python	5.3 %	1
		WED-flow	5.3 %	1
		C	10.5 %	2
		Fortran	10.5 %	2
		Java	10.5 %	2
		Swift/T	10.5 %	2
		Shell Script	15.8 %	3
		Matlab	26.3 %	5
		R	36.8 %	7
		Python	47.4 %	9
		C++	52.6 %	10
Q5	# of People	I conduct research alone	15.8 %	3
		3 people	15.8 %	3
		4 people or more	68.4 %	13
Q6	Joint Research	I do not face difficulties	47.4 %	9
		I face difficulties	52.6 %	10
Q7	Continuing and/ or Reproducing	I do not face difficulties	31.6 %	6
		I face difficulties	68.4 %	13

Table 1 presents the survey results, with both the percentage (%) and the number (#) of respondents. Our results show that most participants have a PhD degree (Q1), worked on more than 10 computational experiments (Q2), and have more than 7 years of experience (Q3). We also found that the most used tools (Q4) are programming and script-based languages (C++, Python, R, Matlab).

In this survey, "experiment" refers to a distinct scientific research study or project carried out using computational tools, not to individual script executions or cluster job runs. Regarding the nature of the experiments, the survey did not collect data on respondents' specific scientific domains; we acknowledge this as a limitation of our survey instrument. The respondents' roles (ranging from undergraduate students to principal investigators) and their preferred tools (Python, R, C++, Matlab) suggest they are engaged in data-intensive computational research across diverse fields. Regarding collaboration aspects, we tried to understand if the respondents usually conduct experiments in groups and the size of those groups. The answers to Q5 indicate that the vast majority conduct experiments in groups, and the groups are typically large (with four or more collaborators).

Although the absence of proper provenance-aware tooling support for collaboration in script-based approaches has been identified as a research opportunity [Jandre *et al*., 2020], a question arises: is this gap relevant from the scientist's point of view? Hence, we had two questions that helped to clarify that: "Do you face any difficulties when performing joint research with other members of your research group?" (Q6) and "Do you face any difficulties when continuing and/or reproducing the work of other researchers?" (Q7). These questions were non-mandatory discursive questions. After we collected the answers, we classified them as "I do not face difficulties" or "I face difficulties" for quantitative purposes; blank answers were considered as "I do not face difficulties."

As shown in Table 1, the majority of respondents face difficulties in collaboration. Moreover, even those who did not have problems with collaboration answered that they face reproducibility problems when trying to continue or reproduce the work of other scientists in the group. Among the respondents who reported difficulties with joint research (Q6), the most frequently mentioned issues were: *communication challenges*, including difficulty expressing research topics clearly and terminology gaps in multidisciplinary groups (e.g., "difficulty in expressing clearly a discussion topic"); *misaligned research interests*, particularly when computer scientists and domain scientists pursue different hypotheses; and *deadline management* difficulties arising from coordination overhead in group work. Among the respondents who reported difficulties with reproducing or continuing the work of others (Q7), the most commonly cited problems were: *environment and dependency issues* (e.g., "instructions to execute a computational activity often do not work due to, for instance, version changes"); *insufficient documentation*; and *difficulty sharing code and data*. Several respondents suggested that better computational environment preservation and tools that facilitate experiment sharing and result communication would help. These results corroborated the gap identified by Jandre *et al*. [2020] and motivated us to look deeply at this particular problem.

3 Related Work

With the growth of in silico experiments, the problem of managing this kind of experiment has received some attention from researchers. Jandre *et al*. [2020] analyzed multiple tools and approaches regarding collaboration in scientific experiments, identifying some gaps. One of them resides in the lack of support for collaboration in provenance-aware script-based approaches. However, Jandre *et al*. [2020] limit themselves to listing the gap – they do not propose an approach to fill it.

Some authors propose provenance models to capture the collaboration in this environment. Missier *et al*. [2010] and Altintas *et al*. [2010, 2011b] present provenance models capable of representing collaborative experiments. Although this is an important subject, scientists still need tools that can collect this data and present it properly to scientists involved in the research. More recently, Coelho *et al*. [2022] proposed a blockchain-based architecture to establish trust among participants in collaborative scientific experimentation. Although it strengthens trust and traceability in collaborations, it does not address the transparent collection of provenance from scripts, which is the focus of our work.

Existing provenance-gathering approaches can be classified as workflow management systems and script-based systems. Workflow management systems provide some collaborative features, while script-based systems provide very few features to support collaboration. Some tools like Confucius [Zhang *et al*., 2012] and VisTrails [Freire *et al*., 2006] present some collaboration support for in silico experiments designed as workflows. Another flavor of workflow-based solution is proposed by AbediniAla and Roy [2022]. They developed an API that can be added to almost any workflow manage-

ment system. The API makes it possible for these workflow systems to behave like version control systems, for example, Git. Each researcher working on the experiment can branch a project/workflow when pulling, and change it locally. When a module in a workflow is replaced, changed, or altered, the API understands it as another version of the workflow. The API lets researchers merge the workflow they changed with the main one when pushing. This process can be reviewed by another researcher before the merge happens. The API has issue tracking and visualization features too, but as good as it is from a collaboration point-of-view, it must be implemented in existing workflow management systems and does not provide support for scripts since it only captures changes in modules, not in their code.

All of these solutions are workflow-based solutions, which demand the experiment to be described in its specific format – they do not support script-based experiments. In fact, scripts are a popular way of representing in silico experiments [Pimentel *et al*., 2019], and thus, supporting them is essential for the scientific community.

AiiDAlab [Yakutovich *et al*., 2021] also lets researchers collect provenance from experiments while supporting collaboration. It focuses on workflows written in Python. However, scientists need to use the AiiDAlab platform to run the experiments. AiiDAlab's platform lets researchers build their workflows and share them with others. The workflows, built with Python, are run remotely on the AiiDAlab's platform, and the provenance is captured there, too. It's possible to download a workflow to run it remotely, as a Virtual Machine image.

There are also other tools that focus on collecting provenance from Python scripts. Servillat *et al*. [2021], Köhler *et al*. [2024] and Salazar *et al*. [2021] developed tools to collect the provenance of astronomical, electrophysiology, and bioinformatics experiments, respectively, because other generic tools do not meet the needs of these specific fields. However, none of them supports collaboration. Other tools, such as YesWorkflow [McPhillips *et al*., 2015], DfAnalyzer [Silva *et al*., 2020], and Komadu [Suriarachchi *et al*., 2015], require the scientist to instrument the code and include provenance-capture directives.

When we evaluate script management, it is natural to consider some software engineering tools like versioning systems. Although versioning tools (e.g., Git [Spinellis, 2012] and Mercurial [O'Sullivan, 2009]) deal with several collaborative aspects of script construction, they are aimed at software development and concerned only with the development phase of script composition. Specific problems of scientific research, such as the analysis phase, reproducibility of the experiment, and the consequent need for provenance collection in the execution phase of the experiment [Herschel *et al*., 2017], cannot be solved using only versioning tools.

Besides that, the more traditional solutions of script-based experiment management systems [Murta *et al*., 2014; Lerner and Boose, 2014] are focused only on collecting provenance and do not deal with collaboration aspects. Even solutions that address collaborative aspects of the experiment, such as ProvDB [Miao *et al*., 2017] and Sumatra [Davison, 2012; Davison *et al*., 2018], still have some limitations. Despite having a feature that allows multiple users to use the tool, Sumatra does not deal with the input and output files of the experiment, and the researcher is in charge of synchronizing this data. Sumatra also requires that the server be accessible during the execution of the experiment, with the risk of losing information if the server is not reachable. ProvDB, on the other hand, has limitations such as: it depends on an external tool (Git); it works only on a specific operating system (UNIX); and it focuses on a specific type of experiment (data science analysis).

Table 2 summarizes the tools discussed in this section, comparing them according to the main criteria used to select the tool to build upon: whether it supports script-based experiments, whether it captures provenance transparently (i.e., without requiring code instrumentation), whether it is open source, whether it supports collaboration among scientists, and whether it supports storing provenance of multiple distinct experiments.

Our proposed approach aims to take advantage of the progress in script-based experiment management systems. To do so, we evaluated well-documented open source tools we could build on and leverage to unleash the collaborative benefits. Several tools were considered: noWorkflow [Pimentel *et al*., 2017], Sumatra [Davison, 2012; Davison *et al*., 2018], YesWorkflow [McPhillips *et al*., 2015], DfAnalyzer [Silva *et al*., 2020], and Komadu [Suriarachchi *et al*., 2015]. Most of them are not transparent to the user, meaning they require users to change the code (by instrumenting it with provenance annotations or similar strategies) to properly capture provenance. When we exclude the non-transparent approaches, we end up with two possibilities: noWorkflow and Sumatra. We chose noWorkflow because, compared to Sumatra, it allows capturing provenance in a finer grain. It also presents a more elaborate visualization and search layers that facilitate the perception of the research evolution and the access to the collected data. In the next section, we provide an overview of noWorkflow aspects that are relevant to our approach.

4 noWorkflow

NoWorkflow [Murta *et al*., 2014; Pimentel *et al*., 2017] is an open source tool capable of collecting, storing, and visualizing provenance from Python scripts. NoWorkflow works transparently because it does not require instrumentation. It analyzes the script execution, capturing and storing provenance information that can later be used in the analysis phase of the experiment. For that, assuming the experiment is written in a Python file named *script.py*, instead of running *python script.py*, scientists simply run *now run script.py*.

The noWorkflow architecture can be divided into three modules (as shown in Figure 1): provenance collection, provenance storage, and provenance analysis. The provenance collection module is responsible for executing the Python script and also for collecting its provenance. The provenance storage module is responsible for storing both input and output files and capturing provenance (executed functions, parameters, variable values, etc.). The provenance analysis module is responsible for providing querying interfaces to the provenance storage. It also provides pre-built graphic visualizations that help scientists to better understand the experiment and its

Table 2. Comparison of tools for managing in silico scientific experiments. "–" indicates the criterion is not applicable or not reported.

Tool	Script-based	Transparent	Open Source	Collab. Support	Multi-exp. Support
Confucius [Zhang *et al*., 2012]	No	–	–	Yes	–
VisTrails [Freire *et al*., 2006]	No	–	Yes	Yes	–
Abediniala et al. [2022]	No	–	–	Yes	–
AiiDAlab [Yakutovich *et al*., 2021]	No	–	Yes	Yes	–
noWorkflow [Murta *et al*., 2014]	Yes	Yes	Yes	No	No
Sumatra [Davison, 2012]	Yes	Yes	Yes	Partial	Partial
RDataTracker [Lerner and Boose, 2014]	Yes	No	Yes	No	No
ProvDB [Miao *et al*., 2017]	Yes	No	Yes	Partial	No
YesWorkflow [McPhillips *et al*., 2015]	Yes	No	Yes	No	No
DfAnalyzer [Silva *et al*., 2020]	Yes	No	Yes	No	No
Komadu [Suriarachchi *et al*., 2015]	Yes	No	Yes	No	No

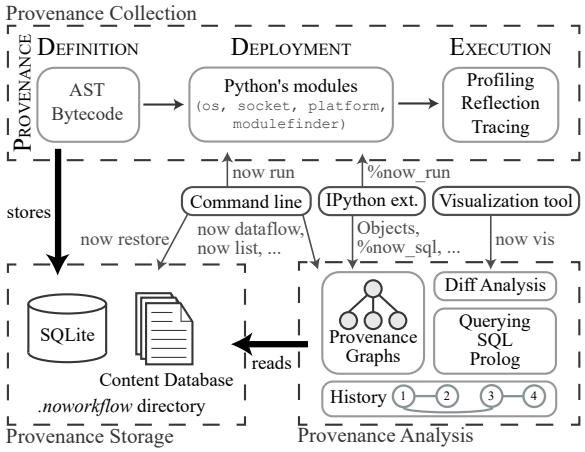


Figure 1. noWorkflow architecture [Pimentel *et al*., 2017].

provenance. We propose extensions that add new collaboration features to the provenance storage and analysis modules. Thus, in this section, we provide details of these modules only.

Provenance Storage. In the provenance storage, data is stored in a folder called *.noWorkflow*, which is automatically created by noWorkflow in the directory where the script execution was invoked (using *now run script.py*). Each script execution corresponds to a *trial* that is stored in the provenance storage. The information collected by noWorkflow is spread across an SQLite database and a content database.

The content database stores all files used during the execution of a trial. This includes the script itself, versions of the used Python libraries, and the input/output files of the experiment (as well as intermediate files). To store these files, noWorkflow first generates the SHA1 hash code of the file content. Then, the first two characters of the SHA1 hash code are used as the folder name, and the remaining characters are used as the file name. For example, a file with SHA1 hash code *2aae6c35c94fcfb415dbe95f408b9ce91ee846ed* will be stored in the folder *./noWorkflow/2a* with file name *ae6c35c94fcfb415dbe95f408b9ce91ee846ed*. Using SHA1 hash codes to identify the files prevents noWorkflow from storing the same file multiple times and makes it simple to identify multiple trials that depend on/use the same files. Additionally, placing the files in folders according to their hash prefix prevents OS limitations on the number of files stored in a single folder.

The SQLite database stores information regarding the execution of the script, such as environment variables, execution parameters, function calls, and other information, depending on the level of granularity chosen by the user in the provenance collection. This database contains a trial table with an auto-increment identifier and several other tables, which are related using foreign keys. The content database and the SQLite database are linked together by the file hash code saved in the SQLite database. Figure 2 shows the database model for noWorkflow's SQLite database.

Provenance Analysis. The provenance analysis module provides several querying options: command line queries, Prolog queries, and SQL queries (which must be run directly in the SQLite database). noWorkflow also has a visualization tool (shown in Figure 3) activated by the command *now vis*. This command starts a web interface that allows viewing the collected provenance, comparing two trials, inspecting accessed files, etc. It is a quick way of checking dependencies, function activations, parameters, etc.

Panel (1) of Figure 3 shows a graph with the trial history. Circles represent an execution of a script, a trial (1.1.1, for instance). In the Figure, there is one execution of the trial 1.1.1, shown in Panel (1). The trial history graph shows that the scientist ran another trial after trial 1.1.1, creating trial 1.1.2. An arrow from circle 1.1.2 to circle 1.1.1 represents the re-execution of the script code. All of this history is captured automatically and transparently by noWorkflow, without the need to use a version control system.

Panels (2) and (3) show details of the trial that is selected in Panel (1). The blue circle around the trial 1.1.1 on Panel (1) means that this is the selected trial detailed in Panels (2) and (3). Panel (2) shows a graph that represents the execution of trial 1.1.1. Each node in this graph corresponds to executing a function in the script (which noWorkflow calls *function activation*). Solid arrows represent a sequence of calls, and dashed arrows represent returns. Panel (2) can also compare the graphs of two trials, showing similarities and differences when two trials are selected in Panel (1). Finally, Panel (3) presents details of trial 1.1.1, such as execution duration, module dependencies, arguments, environment variables, etc.

Lack of collaboration support. NoWorkflow was designed for single users. Consequently, collaboration is not supported by the tool itself, and scientists must do some workarounds to work collaboratively when using noWorkflow. As an ex-

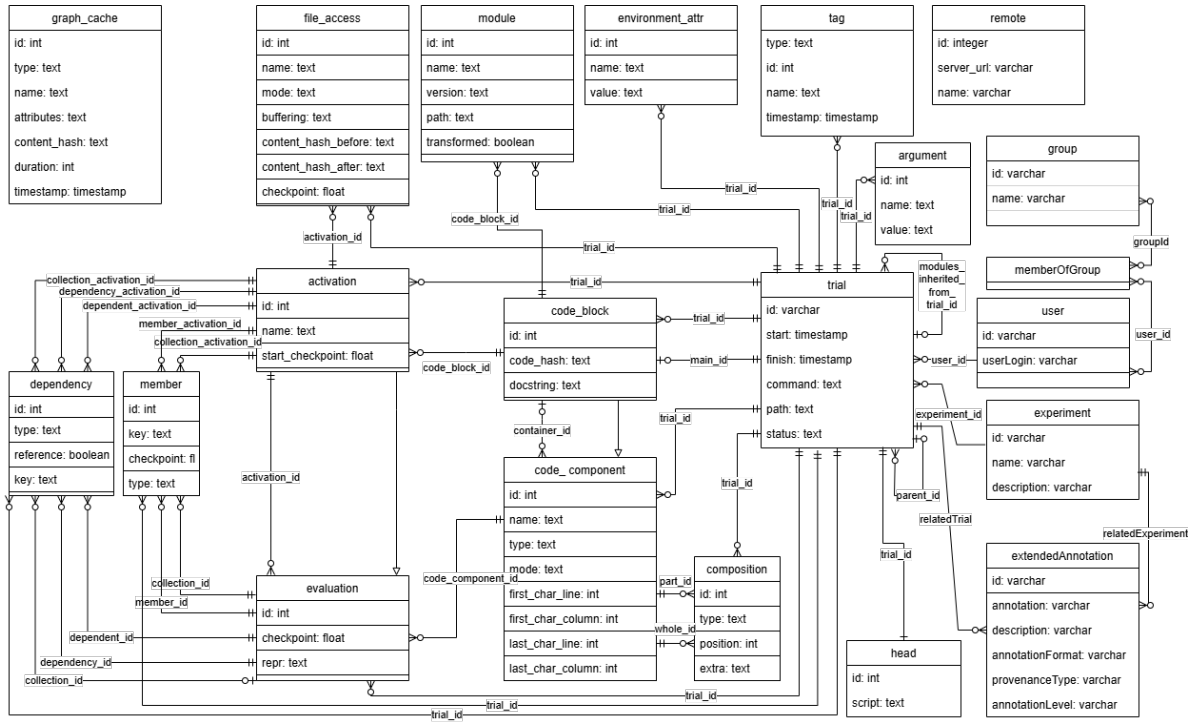


Figure 2. noWorkflow's relational data model.

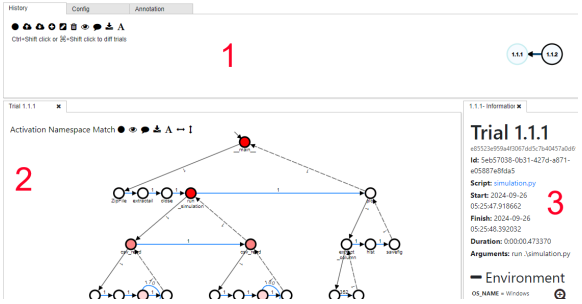


Figure 3. View of the provenance information in noWorflow. (1) Graph with the trial history showing two trials; (2) graph view of trial 1.1.1 – nodes are function activations, and arrows denote sequence of calls (solid arrows) and returns (dashed arrows); (3) Details of trial 1.1.1.

ample, suppose that scientists Alice and Bob are working collaboratively on an experiment. Alice writes the script and runs the first trial. Then, she compresses the *provenance storage* (the .noWorkflow folder) and sends it to Bob by e-mail along with the script of the experiment. Bob then uncompresses the folder, makes some changes to the script, and runs it. He then does the same: compresses the folder and sends it back to Alice. Although this allows them to work collaboratively, it adds the burden of compressing and sending files back and forth, but, most importantly, it prevents Alice and Bob from working in parallel, which is unthinkable in the present days. To address this problem, Alice could save the provenance storage in a shared network folder, but that would imply that users have to be in the same network, and, most importantly, since noWorkflow was not designed to work this way, several concurrency problems may occur. Using a shared cloud storage bucket or driver would incur in the same types of synchronization problems. This is because the provenance database is a single SQLite file. Thus, every time two users run an experiment in parallel, the database file would be overwritten and data would be lost, since no merge mechanism would be in place to take care of synchronizing only the differences.

In the next section, we propose a provenance-aware collaborative approach to experiments designed as scripts, and in Section 5.1, we detail the implementation of this approach on top of noWorkflow.

5 Supporting Collaboration

To support the execution of collaborative experiments, we propose an approach that allows aggregating, synchronizing, and analyzing the provenance information of an experiment conducted collaboratively by several scientists. To achieve that, our proposed approach consolidates the provenance database of the various scientists involved in the experiment.

In our approach, we use concepts of DVCS (Distributed Version Control Systems) [O'Sullivan, 2009; Spinellis, 2012], which are already consolidated in Software Engineering. The idea is that each scientist has their own provenance storage containing the information of the experiment. This repository is then synchronized with a central repository, where it would be available to other interested parties. To achieve these goals, we propose two new operations (*pull* and *push*) to allow the scientists to send and receive data from another provenance storage. Both operations receive an *address* as a parameter, indicating on what provenance storage the operation should be made.

Figure 4 illustrates how the proposed approach works. In this scenario, two scientists work together on the same experiment using a central node as the provenance repository. When one scientist executes the *push* operation, their machine asks the server for the list of trials that are stored on the server. Then, it compares this list with the local provenance database, decides which trials have to be sent to the central node, and

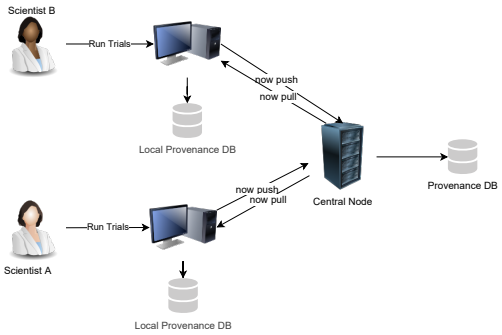


Figure 4. Central node collaboration architecture.

sends them out. The *pull* operation has a very similar behavior, but, in this case, the scientist's machine discovers which trials on the server are not in the local storage and then retrieves them from the server.

This approach allows the scientists to work offline without access to the central node. When back online, scientists can share their work (using the *push* operation) and obtain the progress made by others (using the *pull* operation). This has the additional advantage of centralizing the provenance data of the whole experiment into a single provenance database, which could be very helpful for the scientists working on the experiment. Some of the advantages reside in facilitating scientists to reproduce the trials executed by others, attributing the correct credit to those involved, and making it easier for third parties to understand and reproduce the experiment.

When two scientists push simultaneously, each uploads a distinct set of trials (identified by their UUIDs); since UUIDs are generated independently on each scientist's machine, there is no risk of two scientists generating the same identifier and thus no data overwriting. Concurrent writes to the server database are managed by SQLite's built-in concurrency control. A table in the provenance database records which scientist executed each trial, preserving full attribution.

Note that a code merge does not occur because what is pushed and pulled is the experiment's collected provenance, not the script code itself. Each trial is an independent record identified by a globally unique UUID generated locally at run time. Consequently, if different scientists execute trials originating from the same parent trial, this results in multiple child trials linked to that same parent within the central node's provenance database, all coexisting without conflict. While these child trials could be perceived as branches, they represent distinct script executions with potentially distinct data, duration, and other provenance information that should not be merged.

To illustrate a use case where these operations are used, we can return to the Alice and Bob example of Section 4, but now considering they use our collaboration extension. Alice writes the script and runs the first trial. Then, she uses the *push* command to send provenance information to the central node server. Bob then uses the *pull* command to receive experiment data and then changes the script and runs it. He then does the same, use the *push* command to send information to the central node where it will be available for Alice and other scientists. This will produce the same provenance database as the example explained in Section 4, but in a much easier and more fluid way for scientists. Another important evolution is the capability of working in parallel and still being able to

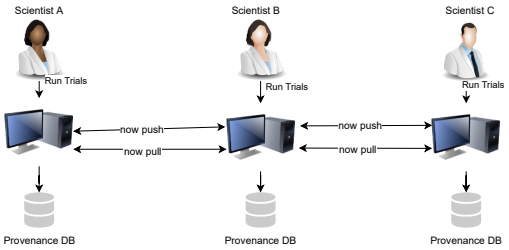


Figure 5. Peer-to-peer collaboration architecture.

consolidate the provenance into a single provenance database, which was not possible before.

Additionally, we propose the support for multiple experiments. This is relevant since the design of some tools that capture the provenance of scripts tends to support a single experiment only (with several trials). New experiments are stored in different provenance databases. The support of multiple experiments, allied with the central node approach, allows a scientific institution or a research group to concentrate the provenance data of multiple experiments into a single provenance storage. This has several uses in increasing collaboration, such as scientists could identify similar experiments, i.e., experiments that use the same dataset or the same algorithms; scientists could search the database to identify software pieces that they can reuse; and also identify implicit collaborations (when one experiment uses another experiment output as input, for example, creating a collaboration even though the scientists involved do not meet or talk to each other).

Although we see much value in having a central node to store provenance data, we also understand that this might not be possible in some scenarios. So, in our approach, we also support a peer-to-peer model, allowing scientists to share data without needing a central infrastructure. This model also uses the *push* and *pull* operations used for the central node approach. As shown in Figure 5, each scientist has their own provenance database in this scenario. If Alice wants to share or retrieve provenance data from Bob, she connects directly with Bob's provenance database using the pull and push commands. After that, Bob can even connect with another scientist, Jack, and transmit the received data.

The ideas we propose here for allowing collaboration could be implemented in any provenance system. Its core consists of:

- a way to identify provenance databases that scientists can use to pull and push provenance;
- pull and push strategies to allow scientists to send and get provenance from a provenance database;
- a diff algorithm for the content database, if any (some approaches use only a metadata database, and do not store copies of the files, so in those cases, this algorithm would not be needed);
- a diff algorithm for the metadata database (in our case, this is a relational database, but it could be a graph database or any other type of database).

In the next section, we show how we implement each of these aspects in the noWorkflow tool.

5.1 Implementation in noWorkflow

As detailed in Section 3, we choose noWorkflow [Murta *et al*., 2014; Pimentel *et al*., 2017] to extend and implement our proposed approach. To implement the *push* and *pull* operations, we built Restful APIs [Richardson *et al*., 2013] on the server (central node) to return/receive experiment data. We then added new commands to the noWorkflow command-line interface to interact with the server APIs and update the local provenance store. Since we are using REST APIs, it was a natural decision to use a URL as the *address* parameter, which indicates the address of the provenance database. Since noWorkflow already has an embedded HTTP application used in the visualization tool (Figure 3), we decided to build on top of that application and use our REST endpoints to extend it. To make it easier to switch between the central node and the peer-to-peer operation models, we decided to use the same code base on the server and the client machines, as well as the same database structure.

As mentioned in Section 4, noWorkflow provenance storage is composed of an SQLite database, which stores structured provenance data, and a content base, which stores file contents. Thus, we split the problem into how to synchronize these two databases.

Synchronizing the Content Database. Starting with the content database, we built three API endpoints: *list files*, *get file content*, and *store files*. To implement the *push* operation, our noWorkflow client application first calls the *list files* endpoint and retrieves all files existing in the server. Then, it compares this list with the local storage, identifying which files exist in the local storage that do not exist on the server. Since noWorkflow uses the hash of the file content as an identifier, this operation is straightforward. With the list of files that have to be sent to the server, the noWorkflow client application calls the *store file* endpoint to store those files.

To implement the *pull* operation, our noWorkflow client application also calls the *list files* endpoint and retrieves all files existing on the server. However, instead of determining "missing" files on the server, it looks for files in the server that do not exist in the local provenance storage. With the list of files that need to be retrieved, it calls the *get file content* endpoint to get file contents and then stores them in the local database.

Considering scientific experiments nowadays, e.g. including large-scale applications and environments, we understand that those files could be large and that this could be a problem. To mitigate that, all endpoints use gzip [Deutsch *et al*., 1996] compression to minimize network traffic. We understand that, in many cases, this may not be enough. Thus, improving this in future versions of our tool is on our radar.

Synchronizing the SQLite Database. The process of synchronizing the SQLite database is similar but more complex. The noWorkflow SQLite database consists of a standard relational database with many entities and relationships whose referential integrity must be preserved. To help implement the commands, we also built three API endpoints: *list trials*, *retrieve trial data*, and *store trial data*. Considering the database is "trial-oriented," we use the trial entity as the starting point of the synchronization.

The process for implementing the *push* operation is similar to the one described for the content database. We determine which trials must be sent to the server and use the *store trial data* to save information on the server. Since noWorkflow uses an auto-increment integer as a trial identifier, it is not easy to identify if a trial on the server is the same as one in the local storage. One approach could be to compare all trial data (including child entities data), but this could be time-consuming. Another critical issue is that the trial table has a self-reference to identify the "trial parent," which could demand a prohibitive comparison effort on synchronization. To deal with that, we have modified the trial table, renaming the trial id to "sequence key" and creating another column as the trial id where we store a UUID (Universally Unique IDentifier) [Leach *et al*., 2005]. We also modified the noWorkflow *run* command to generate this UUID and adapted all foreign keys to the trial table to reflect that change. This identifier is unique globally, and we use it on the *list trials* endpoint and on the noWorkflow client application to infer trials that must be sent to the server. To send trial information to the server, we choose to use the JSON (JavaScript Object Notation) format, using a very standard serialization process to transform query data into JSON format. When the server receives JSON content, it deserializes it and then stores it in the server database.

For the *pull* operation, the noWorkflow client application also calls the *list trials* endpoint and retrieves all trials existing on the server. But now, it determines which trials must be retrieved. Then it calls the *retrieve trial data* endpoint to get the trials' information and then stores it in local provenance storage. The process of comparing trials, serialization, and deserialization is analogous to the explained for the *push* operation.

Support of Multiple Experiments. We also extended noWorkflow to create the concept of multiple experiments using a single provenance storage. We see this as a very important improvement that will allow us to easily compare experiments, understand "hidden" relationships between them, commonly used data, etc. To implement this, we created a table on the SQLite database, representing the experiment entity, and added a foreign key on the trial table referring to the experiment.

In addition, we created a new screen (illustrated in Figure 6), which allows scientists to create new experiments, navigate to experiment provenance details (same screen illustrated in Figure 3), and also list existing experiments and the URL that must be used with *push* and *pull* commands to provenance storage sync. We made this screen to be the home screen of the noWorkflow HTTP application. The multi-experiment feature can also help with the network traffic problem since users will often benefit from a file already being on the server because another user in another experiment has already stored the information there.

Since we have the same source code for the noWorkflow client application and the server, the implementation of the peer-to-peer model uses the same mechanism already explained. In the peer-to-peer model, the scientist just runs the HTTP application embedded on their machine, and it can assume a server role. The only prerequisite is that the other

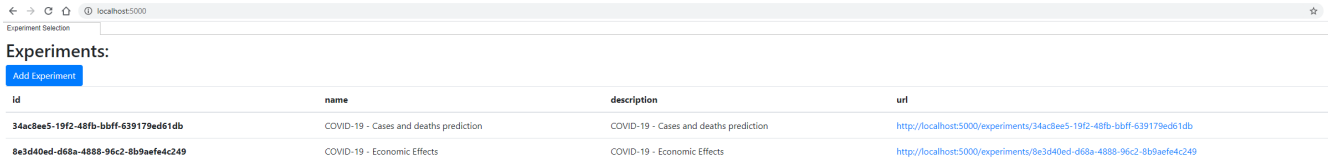


Figure 6. Experiment portal screen

scientist could reach this machine using the HTTP protocol.

6 Experimental Evaluation

To validate our approach, we built a simulated experiment that aims at showing (a) how the described approach can capture information and iterations not captured before, and (b) how this data can be used to produce insights about the experiment. The results are shown in Section 6.1. Then, in Section 6.2, we show that our approach is capable of answering the queries presented at the First Provenance Challenge[2].

6.1 Exploring Collaborative Features

In our first experimental validation, we consider a fictitious research institute called "Diseases Research Institute." This institute has several research groups working on several experiments and uses our approach to centralize the provenance data of those experiments. The experiments cited here are also fictitious, so our focus is on the interactions between the researchers and not on the actual experiment code or results.

With COVID-19 being a serious problem impacting the whole world, the Diseases Research Institute conducted multiple types of research on that subject. Back in 2020, one of those experiments aimed to predict the number of infected people and deaths in a country over time. We call this experiment *Experiment A*. *Experiment A* is conducted by several scientists, but since COVID implied in several lockdown practices, many of those scientists started to work remotely. Conducting this experiment collaboratively in a serialized manner with many scientists in different locations is nearly impossible, so much of the work is done in parallel. But since they are using our proposed approach, they could consolidate provenance information of the work done in a single provenance storage. Hence, the results of *Experiment A* were published, and its provenance data were accessible to other researchers at the Institute, who used its results to conduct other experiments. However, after some time, the "Diseases Research Institute" found out that there was one implementation problem (bug) in the script code used by *Experiment A*. Now the Institute needs to discover other experiments that were built on top of *Experiment A* and could have their results contaminated by *Experiment A*'s problems.

Since researchers use our noWorkflow collaborative extension to run their experiments, the Diseases Research Institute has a centralized database containing interactions inside an experiment and the implicit collaborations between the experiments. To identify which experiments are impacted

[2] `https://openprovenance.org/provenance-challenge/FirstProvenanceChallenge.html`

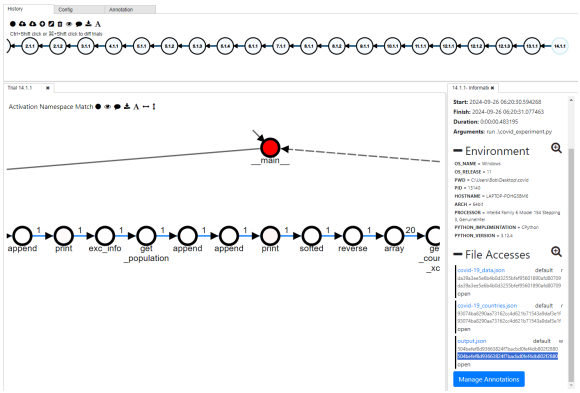


Figure 7. Experiment A provenance visualization

```
SELECT DISTINCT e.id, e.name
FROM trial t, experiment e, file_access a
WHERE t.experiment_id = e.id AND
a.content_hash_after = '504befef8d93663824f7bacbd0fef4db802f2880'
AND t.id = a.trial_id;

QUERY RESULT:
```

id	name
46f7b8a1-8e4a-4f0c-bc3e-fed53db8e2b1	Experiment A - Covid deaths prediction
71d2035c-623e-4013-83c3-75c9d10bddf5	GDP study

Figure 8. SQL query used to find experiments using a specific file

by *Experiment A*, the user entered the experiment portal (illustrated in Figure 6), navigated to *Experiment A* (first line on the experiment list shown in Figure 6), and inspected its outputs (Figure 7). The hash of the produced file is *504befef8d93663824f7bacbd0fef4db802f2880*, as shown in the Figure. Then, the user uses the SQL query illustrated in Figure 8 to query the provenance database and find which experiment used its output.

Using the results shown in Figure 8, the Diseases Research Institute ends up finding one experiment that was impacted by *Experiment A*'s results – an experiment that aims to predict the GDP impact over time caused by COVID-19 on a country. The researchers of the *GDP* experiment must be notified, so the Institute uses the query illustrated in Figure 9 to find and notify people involved in the experiment. Since the GDP experiment was not publicly published yet, they could use the new results of *Experiment A* and correct their experiment.

Another issue is that the Diseases Research Institute has a strict policy for credit attribution, thus demanding that even implicit collaborators should receive credit and be properly registered. To comply with that, the researchers responsible for the GDP experiment used the query illustrated in Figure 10 to find experiments that implicitly collaborate with the GDP experiment. With the results, they can use the query in Figure 9, just adjusting the *experiment_id* parameter, to identify collaborators and give them the proper credit.

Using the noWorkflow current version as a baseline, only one of those three questions could be answered: "Which users

```
SELECT DISTINCT env.value [user]
FROM environment_attr env, trial t, experiment e
WHERE t.id = env.trial_id AND
t.experiment_id = e.id AND
env.name = 'USERNAME' AND
e.id = '71d2035c-623e-4013-83c3-75c9d10bddf5';

QUERY RESULT:
```

user
scientist4
scientist5
scientist6

Figure 9. SQL query used to find scientists involved in experiments.

```
SELECT DISTINCT e.id, e.name
FROM experiment e, trial t, file_access fa
WHERE e.id = t.experiment_id AND
fa.trial_id = t.id AND
EXISTS ( SELECT 1
   FROM experiment e2, trial t2, file_access fa2
   WHERE e2.id = t2.experiment_id AND
   fa2.trial_id = t2.id AND
   e2.id = '71d2035c-623e-4013-83c3-75c9d10bddf5' AND
   e2.id <> e.id AND
   fa2.content_hash_before = fa.content_hash_after
   );
```

Figure 10. SQL query used to find implicit collaborations

are involved in an experiment?". However, to answer this question, the scientists would have to work synchronously and send each other provenance storage over e-mail or share it over a network, for example, which would be error-prone and definitely impact their productivity.

Regarding data transfer performance, the push and pull operations transfer two types of data: provenance metadata (trial records, function call traces, environment information, and file access data), serialized as JSON; and content files (the experiment scripts, input files, and output files captured by noWorkflow). For the case studies presented in this article, the provenance records and scripts are lightweight, as the focus is on the collaborative interactions rather than the data processing volume. The overall transfer time therefore depends on the network speed and the cumulative size of the experiment's files. A systematic analysis of data transfer performance across different experiment sizes and network conditions is planned as future work.

6.2 Answering the First Provenance Challenge

Despite the fact that our experiment was successful when simulating a collaborative environment, we went further to investigate whether the collected provenance meets the researchers' needs. We thus decided to use the First Provenance Challenge[2] and its questions to see if noWorkflow's collected provenance is useful when in a collaborative environment.

However, there are two problems with using the First Provenance Challenge and its questions directly. The first one is that the challenge is about a specific workflow. The keyword being *workflow*, because it's not a Python script like noWorkflow supports. The second one is that the challenge questions are about that specific workflow. Therefore, in this evaluation, we first adapt the questions in the First Provenance Challenge to refer to scripts in general, instead of a specific workflow.

6.2.1 Adapting the Challenge Questions to Scripts

The First Provenance Challenge has nine questions that aim at evaluating the capabilities of workflow systems to capture provenance. Since they were targeted at workflows, in this work, we contribute by adapting some of them to experiments that were written as scripts. For that, we use the following notation.

An experiment e, materialized in the form of a script s is composed of a set of trials T. Each trial $t \in T$ corresponds to a different execution of the script s. The set of trials T of an experiment may correspond to executions of different versions of the script s, including variations of code, parameters, and inputs. A trial t may have zero or more parameters p (these are the parameters that are passed in the command line during the invocation of the script s). We call P the set of all parameters of a trial t ($p \in P$). Also, a trial t may generate zero or more outputs os. We call OS the set of all such outputs ($os \in OS$). We consider an output to be a file that is written by the script.

A trial t has a (possibly empty) set of functions F. A function f of a trial thus belongs to this set ($f \in F$). The functions of a trial t may have zero or more parameters. We call PF the set of all parameters of the functions in F of t. We call pf a given parameter of PF ($pf \in PF$). Additionally, a function f of t may produce a set of outputs (generated via the *return statement*) OSF, where osf is one of such outputs ($osf \in OSF$).

Some of the questions of the First Provenance Challenge refer to a single run of a workflow, in our case, a single run of a trial. Since the purpose of this evaluation is to test noWorkflow *collaboration* capabilities, we decided to focus only on the questions that have answers that can come from more than one trial. Due to that, we discarded questions number one to four (included). So, based on the notation previously described, Table 3 shows the adapted Provenance Challenge questions number five to nine. For all of the adaptations, we took the mentions to the specific workflow defined in the First Provenance Challenge and generalized it as much as possible, while keeping the original query intent. As an example, the fifth question of the challenge is *Find all Atlas Graphic images outputted from workflows where at least one of the input Anatomy Headers had an entry global maximum=4095. The contents of a header file can be extracted as text using the scanheader AIR utility*. All Atlas Graphic images, in this case, are all outputs from executed workflows (i.e., OS). Anatomy Headers are files used as an input in the workflows, and *maximum=4095* is the content that one or more Anatomy Header files could have. In our translation, we consider multiple workflow runs to be equivalent to a set of trails T. An input of a workflow, in our translation, is the same as a parameter p in a script. Thus, the adapted query, as shown in Table 3, reads *Get all the outputs OS of all trials in T that had a parameter p as an input.*.

6.2.2 Answering the Questions

To evaluate whether our extension of noWorkflow can successfully answer those questions, we used the same experiment that was used in the noWorkflow DEMO that was

Table 3. The First Provenance Challenge Queries adapted to scripts

Query	Query change to general script
Q5	Get all the outputs OS of all trials in T that had a parameter p as an input.
Q6	Find all outputs (produced by the return statement) of a function $f, (f \in F)$ if f is preceded by $f', (f' \in F)$ and f' received the parameter pf.
Q7	Given two trials t and t' contemplating the same experiment, find the difference between t and t' executions and functions.
Q8	Find all outputs (produced by the return statement) of a function f, where f received a parameter pf and pf was annotated with a specific annotation a.
Q9	Find all the outputs OS of all trials in T where the outputs were annotated with a specific content ac. For those outputs, return all their other annotations.

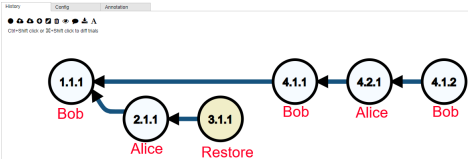


Figure 11. History graph showing who run the trials in our simulation

performed in VLDB in 2017 [Pimentel *et al*., 2017]. Using that code and sequence of trials, we simulate an experiment where two scientists, Bob and Alice, check if the precipitation in Rio de Janeiro remains constant across the years [Pimentel *et al*., 2017].

From the sequence of trials of that demo, we only changed the last two trials. This is because, originally, those were error trials (used to illustrate some functionalities of noWorkflow). For the experiment in this present article, we fixed the code of those trials to make them run successfully. We also added one more trial with parameters to better illustrate Q5's answer.

The steps we took in this simulation are as follows. First, we set up the server and create the experiment on it. Then Bob runs the first trial on his local computer and pushes it to the server. With the experiment and its first trial on the server, Alice pulls it, modifies the code, runs it on her local computer, and finally pushes it to the server. Seeing that there is a new trial on the server, Bob pulls it, but he restores the experiment to the first trial. Then he modifies the code, runs it with a parameter, and pushes it to the server. Alice then pulls the experiment on her local machine. Afterward, she runs the experiment with a different parameter (different from the one Bob used), and she pushes the trial to the server. Finally, Bob pulls the experiment from the server and decides to run another trial with the parameter he used previously. Then he pushes it to the server, ending our simulation.

Figure 11 shows the final history graph, after all the trials were run and pushed to the server, with the name of the person who executed the trial under each circle that represents a trial. For example, Alice ran the trial 2.1.1, so her name is below the trial 2.1.1's circle. The exception is trial 3.1.1, which is a restored trial, and thus nobody executed it.

Using the provenance database generated by noWorkflow, and also noWorkflow's analysis tools, we now answer the First Provenance Challenge questions five to nine. Note that here we instantiate the queries shown in Table 3 to look for specific parts of the provenance, as the original queries of the First Provenance Challenge did.

Q5: Get all the outputs of all trials that had the parameter "2" as an input. We use SQL to answer this query. First, we need the table **file_access** to get the outputs, because it registers the access to files, including writing, meaning the outputs from a trial. We also need to query the table *activation*. This table registers function calls. We must make sure that the file accesses we are querying are the right ones. The table argument must be checked, too. After all, this is the table responsible for checking if the trial received a parameter when it was executed, and which parameter it was.

```
SELECT ta.name as trial_tag, ar.value, f.name,
f.mode, f.content_hash_before, f.content_hash_after
FROM file_access as f, activation as ac, argument as ar
LEFT JOIN tag as ta on ta.trial_id == f.trial_id
WHERE ac.id == f.activation_id AND
ar.trial_id == f.trial_id
AND f.trial_id == ac.trial_id
AND (f.mode == "w" OR f.mode == "w+b")
AND f.name NOT LIKE "nul"
AND ar.name == "argv"
AND ar.value LIKE "%'2'%"
```

This query can be modified to include additional information to expose collaborative aspects. For example, we may want to know who ran the experiment(s), how, and/or when. We can get most of this information by adding the *trial* table to the query. Using our simulation's database, the result for the query that answers Q5 is shown by Table 4 while Table 5 shows the additional information we may get if we modify the query to include some collaborative aspects. Note that, in this case, all trials were executed by Bob.

```
SELECT ta.name as trial_tag, ar.value, f.name, f.mode,
f.content_hash_before, f.content_hash_after, t.script,
t.start, t.finish, t.command, t.path, t.user_id
FROM file_access as f, activation as ac, argument as ar,
trial as t
LEFT JOIN tag as ta on ta.trial_id == f.trial_id
WHERE ac.id == f.activation_id
AND ar.trial_id == f.trial_id
AND f.trial_id == ac.trial_id
AND t.id == ar.trial_id
AND t.id == f.trial_id
AND t.id == ac.trial_id
AND (f.mode == "w" OR f.mode == "w+b")
AND f.name NOT LIKE "nul"
AND ar.name == "argv"
AND ar.value LIKE "%'2'%"
```

Q6: Find all outputs of the function sum_by_month(d12, months) if it is preceded by a read function with parameter p12.dat. This query searches all trials, and so all trials of Bob and Alice need to be searched. To solve this query, we use noWorkflow as a library in a Python script. The resulting script can be found in our repository[3], and the script output is as follows.

```
Found F:  sum_by_month(d12, months)  !
Found G:  read("p12.dat")  !
Found: 'p12.dat' !
```

[3] https://github.com/dew-uff/noworkflow-collab-experiment/blob/main/provenance_challenge_6_with_noworkflow.py

Table 4. Result of the SQL query used to answer Q5 in our simulation

trial_tag	value	name	mode	content_hash_before	content_hash_after
4.1.1	['experiment.py', '2']	temp13.dat	w	0521e41206be4d5e93bd801022ad4ef1d58dc885	f628bd3839d9fc425de54c1fe218bd5115fbb617
4.1.1	['experiment.py', '2']	temp14.dat	w	daa11126d3ae358613cd0e5d53f47c5d17bec73d	daa11126d3ae358613cd0e5d53f47c5d17bec73d
4.1.1	['experiment.py', '2']	temp13.dat	w	f628bd3839d9fc425de54c1fe218bd5115fbb617	0521e41206be4d5e93bd801022ad4ef1d58dc885
4.1.1	['experiment.py', '2']	temp14.dat	w	daa11126d3ae358613cd0e5d53f47c5d17bec73d	daa11126d3ae358613cd0e5d53f47c5d17bec73d
4.1.1	['experiment.py', '2']	C:\Projetos\annual_precipitation_expirement_2\user1\out.png	w+b	a53a6e03f86799fc23b0a119462ba122758ba0df	a53a6e03f86799fc23b0a119462ba122758ba0df
4.1.2	['experiment.py', '2']	temp13.dat	w	0521e41206be4d5e93bd801022ad4ef1d58dc885	f628bd3839d9fc425de54c1fe218bd5115fbb617
4.1.2	['experiment.py', '2']	temp14.dat	w	daa11126d3ae358613cd0e5d53f47c5d17bec73d	daa11126d3ae358613cd0e5d53f47c5d17bec73d
4.1.2	['experiment.py', '2']	temp13.dat	w	f628bd3839d9fc425de54c1fe218bd5115fbb617	0521e41206be4d5e93bd801022ad4ef1d58dc885
4.1.2	['experiment.py', '2']	temp14.dat	w	daa11126d3ae358613cd0e5d53f47c5d17bec73d	daa11126d3ae358613cd0e5d53f47c5d17bec73d
4.1.2	['experiment.py', '2']	C:\Projetos\noworkflow-collab-experiment\user1\out.png	w+b	a53a6e03f86799fc23b0a119462ba122758ba0df	a53a6e03f86799fc23b0a119462ba122758ba0df

Table 5. Result of additional collaboration information to answer Q5 in our simulation

script	start	finish	command	path	user_id
experiment.py	2024-07-18 11:20:37.588015	2024-07-18 11:20:41.009032	run -b experiment.py 2	C:\Projetos\annual_precipitation_expirement_2\user1	LAPTOP-POHG5BM6\Bob
experiment.py	2024-07-18 11:20:37.588015	2024-07-18 11:20:41.009032	run -b experiment.py 2	C:\Projetos\annual_precipitation_expirement_2\user1	LAPTOP-POHG5BM6\Bob
experiment.py	2024-07-18 11:20:37.588015	2024-07-18 11:20:41.009032	run -b experiment.py 2	C:\Projetos\annual_precipitation_expirement_2\user1	LAPTOP-POHG5BM6\Bob
experiment.py	2024-07-18 11:20:37.588015	2024-07-18 11:20:41.009032	run -b experiment.py 2	C:\Projetos\annual_precipitation_expirement_2\user1	LAPTOP-POHG5BM6\Bob
experiment.py	2024-07-18 11:20:37.588015	2024-07-18 11:20:41.009032	run -b experiment.py 2	C:\Projetos\annual_precipitation_expirement_2\user1	LAPTOP-POHG5BM6\Bob
experiment.py	2024-07-18 12:08:51.158671	2024-07-18 12:08:57.596722	run -b experiment.py 2	C:\Projetos\noworkflow-collab-experiment\user1	LAPTOP-POHG5BM6\Bob
experiment.py	2024-07-18 12:08:51.158671	2024-07-18 12:08:57.596722	run -b experiment.py 2	C:\Projetos\noworkflow-collab-experiment\user1	LAPTOP-POHG5BM6\Bob
experiment.py	2024-07-18 12:08:51.158671	2024-07-18 12:08:57.596722	run -b experiment.py 2	C:\Projetos\noworkflow-collab-experiment\user1	LAPTOP-POHG5BM6\Bob
experiment.py	2024-07-18 12:08:51.158671	2024-07-18 12:08:57.596722	run -b experiment.py 2	C:\Projetos\noworkflow-collab-experiment\user1	LAPTOP-POHG5BM6\Bob
experiment.py	2024-07-18 12:08:51.158671	2024-07-18 12:08:57.596722	run -b experiment.py 2	C:\Projetos\noworkflow-collab-experiment\user1	LAPTOP-POHG5BM6\Bob

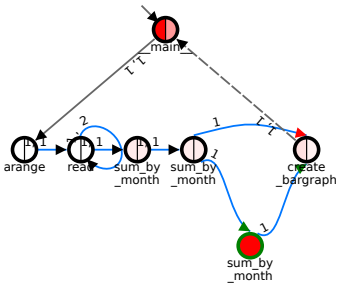


Figure 12. Comparing the function activations between the trials "1.1.1" and "2.1.1"

```
F output:  [220.9, 62.8, 84.10000000000001, 45.4, 128.8,
71.10000000000001, 26.2, 17.9, 87.9, 66.4, 93.2, 37.0]
```

Q7: Find the difference between the execution and functions of trials "1.1.1" and "2.1.1." These trials were conducted by two different scientists, so the previous version of noWorkflow would not be able to answer this query unless Alice manually sent her provenance database to Bob before he did his alterations and run his trial. There are two ways to see the difference between two trials using noWorkflow. One of them is using the command line inside the experiment (or server) folder by executing the command *now diff [trial1_id] [trial2_id]*, where trial1_id and trial2_id are the ids of the trials the user wishes to compare. It's also possible to add other arguments if the user wishes to compare other things. For example, *-m*, *-e*, and *-f* if they want to compare modules, environments, and file accesses, of the trials, respectively.

The second method to compare two trials is by using noWorkflow's graphical interface. Once the user is in the graphical interface of an experiment, they can right-click on the first trial they want to compare, then crtl(control)+click on the other trial. Now, noWorflow then shows the differences between the trials and their functions. Figures 12 and 13 show the result. Notice in Figure 13 the TMP variable that shows evidence of different users executing each trial. The *username* variable also shows that each trial was executed by a different user, but this is not shown in the Figure due to space constraints.

Q8: Find all outputs of the function sum_by_month which received d12 as a parameter and was annotated with "2012 data". Currently, noWorkflow cannot answer this query.

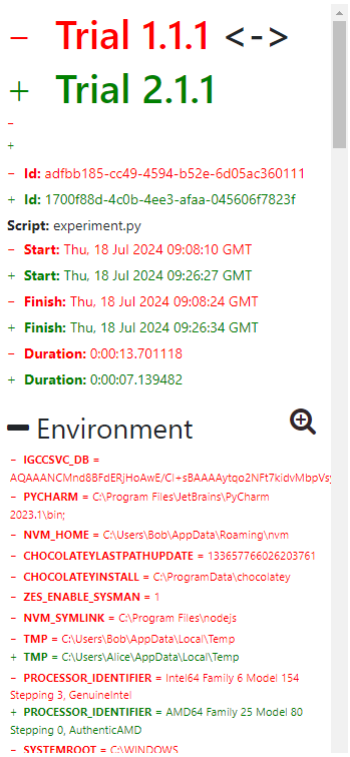


Figure 13. Some differences between the trials "1.1.1" and "2.1.1"

This is because it currently supports annotations in the experiments, but not on individual files.

Q9: Find all the outputs of all trials where the outputs were annotated with "2012 data". Return all the other annotations of all outputs that fulfill the requirement. Since this query also requires annotations in files, noWorkflow also can not answer it.

7 Conclusion

Collaboration is part of science and scientific research. Although this has been going on for centuries, we can see that it is still a growing movement nowadays. One of the catalysts of this has been the advances in computer technology and networks, the same advances that helped to enable and popularize in silico experiments. Although the use of script-based approaches in collaborative research is a common scenario,

there are some gaps in those approaches [Jandre *et al*., 2020].

In this article, we propose a collaborative approach to in silico experiments based on scripts. It allows scientists to work with a central repository, or use several local repositories in a peer-to-peer fashion. We did two validation experiments. The first one uses a hypothetical but very feasible situation. The second one uses the provenance database of an experiment conducted collaboratively to answer the queries of the First Provenance Challenge. To do that, we also contribute by proposing a translation of the queries (that were originally targeted at workflow systems) to experiments that were written as scripts. Those adapted questions can now be used to evaluate other script-based provenance tools.

Our experiments illustrate that our proposed extension of noWorkflow is successfully able to store and query the provenance of a single collaborative experiment and also of several different experiments. The main benefits of our approach, evidenced by our experiments, are: to easily allow collaboration in scientific experiments; to have single provenance storage that stores information about all experiments and all trials of a single experiment, even those run by different scientists; to find the hidden collaboration between scientists and experiments; and more reproducible and auditable experiments. We highlight that principles behind the benefits of our proposal can be applied to other provenance tools. These principles only need to be implemented consistently.

Although we argue that our proposal provides significant improvements in the current scenario, we understand there are some limitations. First, our survey has a limited number of respondents, although it corroborates claims of related literature [Jandre *et al*., 2020; Wuchty *et al*., 2007; Herschel *et al*., 2017]. Second, in silico experiments can produce many data, and transferring this data over the network and even storing data in a distributed manner could be a problem in some big data scenarios. Third, the implementation of control access management is still pending. Finally, our proposal does not consider any security and privacy features, leaving it out of our scope. If necessary and possible, to handle these problems, it should be delegated to the server or peer-to-peer connection when using our current proposal.

Our evaluation also has limitations. We compare the collaborative extension of noWorkflow against its original version, focusing on the new collaborative features introduced by our extension. A direct comparison with other script-based tools such as Sumatra is non-trivial, as those tools do not support the same operations (e.g., sharing individual trials, synchronizing content files, and managing multiple experiments in a single provenance store). We acknowledge this as a limitation and plan a broader comparative evaluation as future work.

As future work, we plan to propose better file storage and data transfer strategies and implement access control management features, since many pieces of research could involve secrecy. Another possibility we want to explore is to apply a similar approach of distributed provenance collection to experiments that run on a distributed infrastructure and not necessarily by multiple scientists. Finally, we plan to conduct a validation experiment with a real use case scenario.

Regarding the noWorkflow core (not directly related to collaboration), we plan to improve it so that using it to capture provenance of Big Data and High-Performance Computing experiments is viable. We are currently working on improving the performance of the provenance capture mechanism of noWorkflow. In addition, we also plan to develop a migration tool for experiments that were made with noWorkflow's previous versions. Currently, the databases of those versions are incompatible, and so data that was captured using the first version cannot be explored using the current version.

Disclaimer

We have used Claude Opus 4.7 and Grammarly to sharpen the text of this article. We have carefully revised each of their suggestions and thus assume full responsibility for the article's content.

Funding

This research was funded by CNPq grants 409939/2018-8, 305020/2019-6, and 309410/2023-1, and FAPERJ grant E-26/210.478/2024.

Authors' Contributions

Eduardo proposed and implemented the collaborative extension of noWorkflow and conducted the first literature review. Arthur implemented several improvements to the graphical interface of noWorkflow, proposed the translation of the First Provenance Challenge queries, and answered the queries. He also updated the literature review. João Felipe and Bruna supervised the research and revised the article. Vanessa worked on translating the queries of the First Provenance Challenge, supervised the research, and revised the article. All authors read and approved the final manuscript.

Competing interests

The authors declare that they have no competing interests.

Availability of data and materials

The datasets generated and/or analyzed during the current study are available at `https://github.com/dew-uff/noworkflow-collab-experiment`.

References

AbediniAla, M. and Roy, B. (2022). Facilitating Asynchronous Collaboration in Scientific Workflow Composition Using Provenance. *Proceedings of the ACM on Human-Computer Interaction*, 6(EICS):1–26.

Altintas, I., Anand, M. K., Crawl, D., Bowers, S., Belloum, A., Missier, P., Ludäscher, B., Goble, C. A., and Sloot, P. M. A. (2010). Understanding Collaborative Studies through Interoperable Workflow Provenance. In McGuinness, D. L., Michaelis, J. R., and Moreau, L., editors, *Provenance and Annotation of Data and Processes*, pages 42–58, Berlin, Heidelberg. Springer Berlin Heidelberg.

Altintas, I., Anand, M. K., Vuong, T. N., Bowers, S., Ludäscher, B., and Sloot, P. M. A. (2011a). A Data Model for Analyzing User Collaborations in Workflow-Driven eScience. *International Journal of Computers and Their Applications*, 18:160–179.

Altintas, I., Anand, M. K., Vuong, T. N., Bowers, S., Ludäscher, B., and Sloot, P. M. A. (2011b). A Data Model for Analyzing User Collaborations in Workflow-Driven eScience. *International Journal of Computers and Their Applications*, 18:160–179.

Bennis, W. G. and Biederman, P. W. (2007). *Organizing Genius: The Secrets of Creative Collaboration*. Basic Books.

Caldwell, R. and Lindberg, D. (2018). The social side of science: A human and community endeavor. *Understanding Science 101*.

Coelho, R., Braga, R., David, J. M. N., Stroele, V., Campos, F., and Dantas, M. (2022). A blockchain-based architecture for trust in collaborative scientific experimentation: A blockchain-based architecture for trust in collaborative scientific experimentation. *Journal of Grid Computing*, 20(4):35.

Davidson, S. B. and Freire, J. (2008). Provenance and Scientific Workflows: Challenges and Opportunities. In *ACM Special Interest Group on Management of Data*, pages 1345–1350. ACM.

Davison, A. (2012). Automated Capture of Experiment Context for Easier Reproducibility in Computational Research. *Computing in Science & Engineering*, 14(4):48–56.

Davison, A. P., Mattioni, M., Samarkanov, D., and Teleńczuk, B. (2018). Sumatra: a toolkit for reproducible research. In *Implementing reproducible research*, pages 57–78. Chapman and Hall/CRC.

Deutsch, P. *et al*. (1996). GZIP file format specification version 4.3.

Ellkvist, T., Koop, D., Anderson, E. W., Freire, J., and Silva, C. (2008). Using Provenance to Support Real-Time Collaborative Design of Workflows. In *International Workshop on Provenance and Annotation*, pages 266–279. Springer.

Freire, J., Koop, D., Santos, E., and Silva, C. T. (2008). Provenance for computational tasks: A survey. *Computing in Science & Engineering*, 10(3):11–21.

Freire, J., Silva, C. T., Callahan, S. P., Santos, E., Scheidegger, C. E., and Vo, H. T. (2006). Managing Rapidly-Evolving Scientific Workflows. In Moreau, L. and Foster, I., editors, *International Provenance and Annotation Workshop*, Lecture Notes in Computer Science, pages 10–18. Springer Berlin Heidelberg.

Han, R., Zheng, M., Byna, S., Tang, H., Dong, B., Dai, D., Chen, Y., Kim, D., Hassoun, J., and Thorsley, D. (2024). Prov-io^{+}+: A cross-platform provenance framework for scientific data on hpc systems. *IEEE Transactions on Parallel and Distributed Systems*, 35(5):844–861. DOI: 10.1109/TPDS.2024.3374555.

Herschel, M., Diestelkämper, R., and Lahmar, H. B. (2017). A Survey on Provenance: What for? What form? What from? *The VLDB Journal*, 26(6):881–906.

Jandre, E., Diirr, B., and Braganholo, V. (2020). Provenance in Collaborative in Silico Scientific Research: a Survey. *ACM SIGMOD Record*, 49(2):36–51.

Köhler, C. A., Ulianych, D., Grün, S., Decker, S., and Denker, M. (2024). Facilitating the Sharing of Electrophysiology Data Analysis Results through in-depth Provenance Capture. *eNeuro*, 11(6).

Leach, P., Mealling, M., and Salz, R. (2005). A Universally Unique IDentifier (UUID) URN Namespace. *RFC 4122 (Proposed Standard)*.

Lerner, B. and Boose, E. (2014). RDataTracker: collecting provenance in an interactive scripting environment. In *USENIX Workshop on the Theory and Practice of Provenance (TaPP)*, Cologne. USENIX Association.

McPhillips, T., Song, T., Kolisnik, T., Aulenbach, S., Belhajjame, K., Bocinsky, K., Cao, Y., Cheney, J., Chirigati, F., Dey, S., Freire, J., Jones, C., Koop, D., Missier, P., Schildhauer, M., Schwalm, C., Gil, Y., Hanson, J., and Ludäscher, B. (2015). YesWorkflow: A user-oriented, language-independent tool for recovering workflow information from scripts. *International Journal of Digital Curation*, 10(1):298–313. DOI: 10.2218/ijdc.v10i1.370.

Miao, H., Chavan, A., and Deshpande, A. (2017). ProvDB: Lifecycle Management of Collaborative Analysis Workflows. In *Workshop on Human-In-the-Loop Data Analytics*, pages 7:1–7:6, New York, NY, USA. ACM. DOI: 10.1145/3077257.3077267.

Missier, P., Ludascher, B., Bowers, S., Dey, S., Sarkar, A., Shrestha, B., Altintas, I., Anand, M., and Goble, C. (2010). Linking Multiple Workflow Provenance Traces for Interoperable Collaborative Science. In *Workshop on Workflows in Support of Large-Scale Science*, pages 1–8.

Murta, L., Braganholo, V., Chirigati, F., Koop, D., and Freire, J. (2014). noWorkflow: Capturing and Analyzing Provenance of Scripts. In *International Provenance and Annotation Workshop*, pages 1–12.

O'Sullivan, B. (2009). *Mercurial: The Definitive Guide*. O'Reilly Media, Inc.

Pimentel, J. F., Freire, J., Murta, L., and Braganholo, V. (2019). A Survey on Collecting, Managing, and Analyzing Provenance from Scripts. *ACM Computing Surveys*, 52(3):47:1–47:38.

Pimentel, J. F., Murta, L., Braganholo, V., and Freire, J. (2017). noWorkflow: a Tool for Collecting, Analyzing, and Managing Provenance from Python Scripts. *Proceedings of the VLDB Endowment*, 10(12):1841–1844.

Richardson, L., Amundsen, M., Amundsen, M., and Ruby, S. (2013). *RESTful Web APIs: Services for a Changing World*. O'Reilly Media, Inc.

Salazar, V. W., Cavalcante, J. V. F., de Oliveira, D., Thompson, F., and Mattoso, M. (2021). BioProv-A Provenance Library for Bioinformatics Workflows. *Journal of Open Source Software*, 6(67):3622.

Sawyer, K. (2017). *Group genius: The creative power of collaboration*. Basic books.

Servillat, M., Bonnarel, F., Boisson, C., Louys, M., Ruiz, J. E., and Sanguillon, M. (2021). Towards a Provenance Management System for Astronomical Observatories. In *Provenance and Annotation of Data and Processes*, pages 244–249. Springer.

Silva, V., Campos, V., Guedes, T., Camata, J. J., de Oliveira, D., Coutinho, A. L. G. A., Valduriez, P., and Mattoso, M. (2020). Dfanalyzer: Runtime dataflow analysis tool for computational science and engineering applications. *SoftwareX*, 12:100592. DOI: 10.1016/J.SOFTX.2020.100592.

Sonnenwald, D. H. (2007). Scientific Collaboration. *An-*

nual Review of Information Science and Technology, 41(1):643–681.

Spinellis, D. (2012). Git. *IEEE Software*, 29(3):100–101.

Suriarachchi, I., Zhou, Q., and Plale, B. (2015). Komadu: A capture and visualization system for scientific data provenance. *Journal of Open Research Software*, 3(1):e4. DOI: 10.5334/jors.bq.

Tanenbaum, A. S. (2002). *Computer Networks*. Prentice Hall.

Travassos, G. H. and Barros, M. O. (2003). Contributions of In Virtuo and In Silico Experiments for the Future of Empirical Studies in Software Engineering. In *Workshop on Empirical Software Engineering the Future of Empirical Studies in Software Engineering*, pages 117–130.

Wuchty, S., Jones, B. F., and Uzzi, B. (2007). The Increasing Dominance of Teams in Production of Knowledge. *Science*, 316(5827):1036–1039.

Yakutovich, A. V., Eimre, K., Schütt, O., Talirz, L., Adorf, C. S., Andersen, C. W., Ditler, E., Du, D., Passerone, D., Smit, B., *et al*. (2021). AiiDAlab – An Ecosystem for Developing, Executing, and Sharing Scientific Workflows. *Computational Materials Science*, 188:110165.

Zhang, J. (2010). Co-Taverna: A Tool Supporting Collaborative Scientific Workflows. In *IEEE International Conference on Services Computing*, pages 41–48. Institute of Electrical and Electronic Engineers. DOI: 10.1109/SCC.2010.99.

Zhang, J., Kuc, D., and Lu, S. (2012). Confucius: A tool supporting collaborative scientific workflow composition. *IEEE Transactions on Services Computing*, 7(1).